Detección de fauna en cámaras trampa con YOLO: generalización a ubicaciones nuevas y monitoreo ecológico en CCT-20

Grupo 6: Contreras Avendaño, Juan Sebastian Francesco; Romero Lovera, Junior; Muñoz Cañabe, Luis; Oha Quispe, Derly Raúl; Pedraza Palomino, Lourdes
Facultad de Ingeniería de Sistemas e Industrial – Universidad Nacional de Ingeniería
Redes Neuronales y Aprendizaje Profundo, 2026-II

31 de octubre de 2026

Resumen

Las cámaras trampa generan millones de fotos, en su mayoría vacías, y revisarlas a mano es el cuello de botella del monitoreo de fauna. Ajustamos YOLOv8 sobre Caltech Camera Traps (CCT-20) y evaluamos por separado en cámaras vistas en entrenamiento (*cis*) y en cámaras nuevas (*trans*). Agregar imágenes vacías de otras ubicaciones como fondo reduce los falsos disparos a la mitad y mejora el mAP. El modelo final alcanza mAP@0.5 de 0,721 en *cis* y 0,459 en *trans*; la caída es sobre todo de recall y se concentra en especies que en entrenamiento aparecen en pocas cámaras. Una prueba de borrado del animal descarta que el detector prediga especies a partir del fondo, mientras que EigenCAM sobre el *backbone* muestra que la representación sí está dominada por la escena. Por último, un módulo de monitoreo basado en eventos, aplicado a una cámara nunca vista, reproduce los patrones de actividad nocturna de las especies comunes, pero pierde el 19 % de los eventos con animal y casi no registra especies pequeñas.

1. Introducción

Una cámara trampa se dispara por movimiento o calor y deja una ráfaga de fotos. En proyectos de monitoreo es habitual acumular cientos de miles de imágenes por temporada, buena parte de ellas disparadas por viento, sombras o cambios de luz y sin ningún animal. Los detectores de objetos prometen filtrar las vacías y clasificar las especies, pero su desempeño suele reportarse en las mismas cámaras con las que se entrenaron. Beery et al. [1] mostraron que en ubicaciones nuevas el rendimiento cae de forma marcada, porque el fondo, el ángulo y la iluminación de cada cámara son casi constantes y el modelo puede apoyarse en ellos.

En este trabajo estudiamos ese problema con un caso práctico: ajustar YOLO a CCT-20 y preguntarnos qué tan útil sería en una cámara recién instalada. Las contribuciones son: (i) un experimento controlado sobre el uso de imágenes vacías como fondo; (ii) un análisis del AP por especie frente al desbalance de clases; (iii) dos pruebas complementarias (borrado del animal y EigenCAM) para distinguir entre "aprendió el fondo" y "no reconoce al animal en un fondo nuevo"; (iv) una medida de robustez por tamaño de caja y por oclusión etiquetada manualmente; y (v) un módulo de monitoreo basado en reglas, evaluado contra la revisión manual en una cámara no vista.

2. Trabajo relacionado

Norouzzadeh et al. [8] demostraron que las CNN pueden identificar especies en Snapshot Serengeti con precisión cercana a la humana, y Tabak et al. [10] llevaron la idea a modelos listos para usar en ecología. MegaDetector [2] cambió el enfoque: un detector genérico de "animal / persona / vehículo" que filtra vacías y deja la especie a otro modelo. No lo usamos porque no da la especie y porque entre sus datos de entrenamiento

figura Caltech Camera Traps, lo que contaminaría la evaluación *trans*. CCT-20 fue publicado precisamente para medir la generalización a ubicaciones nuevas [1], y el fenómeno de fondo que estudiamos es un caso del aprendizaje por atajos descrito por Geirhos et al. [4]. Para interpretabilidad partimos de Grad-CAM [9] y usamos su variante sin gradientes, EigenCAM [7], más fácil de aplicar a un detector.

3. Datos

Usamos CCT-20 [1]: 57 864 fotos de 20 cámaras del suroeste de Estados Unidos con 16 categorías (14 especies, *car* y *empty*), en la versión reducida a 1024 px de lado mayor. Los splits oficiales separan las cámaras: `train`, `cis_val` y `cis_test` comparten las mismas 10 cámaras, mientras que `trans_val` (1 cámara) y `trans_test` (9 cámaras) usan ubicaciones que no aparecen en entrenamiento. Comprobamos que no hay cruce de ubicaciones entre `train` y `trans_test`.

La exploración mostró tres rasgos que condicionaron las decisiones. **(1)** El split `train` no tiene ninguna imagen vacía, aunque en test el 7–8 % de las fotos lo son. **(2)** El desbalance es extremo: zarigüeya tiene 2 514 cajas de entrenamiento y zorro, 5. Además, cada especie suele concentrarse en pocas cámaras; por ejemplo, el 74 % de los gatos de train proviene de una sola. **(3)** Entre el 55 y el 72 % de las fotos son nocturnas (infrarrojas, en escala de grises), y alrededor del 10 % de las cajas ocupan menos del 1 % de la imagen.

4. Método

4.1. Preparación del conjunto de detección

Exclusiones. Hay 1 454 imágenes de train (y varios miles en test) etiquetadas con una especie pero sin caja. No sirven como positivo, porque no hay caja que aprender, ni como fondo, porque le enseñarían al modelo a ignorar un animal presente; por eso se excluyen de todos los splits. También excluimos la clase *car*, que no es fauna, junto con sus imágenes. Las cajas se recortan a los bordes de la imagen y se convierten a formato YOLO. El conjunto final tiene 11 431 imágenes de train, 4 213 de validación (`cis_val` + `trans_val`), 13 085 de `cis_test` y 18 698 de `trans_test`, con 14 especies.

Vacías como fondo. Para compensar la ausencia de vacías en train, descargamos 3 000 fotos vacías de 106 ubicaciones de Caltech Camera Traps que no forman parte de CCT-20, repartidas por ubicación para no sesgar hacia una sola escena. En YOLO, una imagen con archivo de etiquetas vacío actúa como ejemplo negativo. Usamos 2 286 (20 % de las imágenes con animal), una proporción que deja dominar a los positivos sin dejar de penalizar los falsos disparos. Al venir de otras ubicaciones, no filtran información de las cámaras de validación ni de test.

4.2. Entrenamiento

Partimos de pesos preentrenados en COCO [6] y usamos la implementación de Ultralytics [5] (v8.4.192, PyTorch 2.11, una GPU Tesla T4 en Google Colab). Las decisiones principales fueron:

- **Resolución 640 px.** Es el tamaño nativo de los pesos COCO y cabe en la T4 con lotes de 16–32. Se consideró 960 px por las cajas pequeñas, pero el costo de cómputo se duplicaba y se dejó como trabajo futuro.
- **Aumentos.** Mosaico (desactivado en las últimas 5 épocas para que el modelo vea imágenes completas), volteo horizontal ($p = 0{,}5$), sin volteo vertical porque los animales no aparecen de cabeza, y saturación reducida ($h_s = 0{,}4$ frente a 0,7 por defecto), ya que más de la mitad de las fotos son infrarrojas y una saturación agresiva genera colores que nunca ocurren.
- **Parada temprana** con paciencia de 10 épocas sobre el mAP de validación y semilla fija (42).

Realizamos tres experimentos: E1 (YOLOv8n, sin vacías, 30 épocas) y E2 (idéntico, con vacías) para aislar el efecto del fondo; y E3 (YOLOv8s, 50 épocas, con la mejor opción de E1/E2) como modelo final. El

umbral de confianza para los análisis por caja se eligió en validación con la curva F1–confianza (Apéndice A), nunca en test.

4.3. Evaluación

Reportamos mAP@0.5 y mAP@0.5:0.95 globales y por especie en `cis_test` y `trans_test`. Como métrica propia, definimos la *tasa de falsos disparos*: el porcentaje de imágenes vacías con al menos una detección de confianza $\geq 0{,}5$, que es lo que en la práctica decide cuántas fotos vacías llegarían al revisor humano.

Prueba de borrado. Sobre 300 imágenes con un solo animal por split, cubrimos la caja (con un margen del 15 %) con el color medio de la imagen y volvemos a detectar. Si la misma especie reaparece en la zona borrada o en otra parte de la escena, la decisión se apoya en el contexto y no en el animal.

EigenCAM. Grad-CAM [9] necesita un único puntaje del que derivar gradientes, algo que un detector con cientos de cajas candidatas no tiene de forma natural. EigenCAM [7] evita ese problema: proyecta las activaciones de una capa sobre su primera componente principal. Lo implementamos sobre la salida del bloque SPPF (capa 9, final del *backbone*) y, además de las figuras, calculamos el *lift*: la fracción de la energía del mapa que cae dentro de las cajas reales dividida entre la fracción de área que ocupan. Un lift de 1 significa que el mapa no prefiere al animal sobre el resto de la imagen.

Tamaño y oclusión. Para cada caja real calculamos si fue encontrada con IoU $\geq 0{,}5$ y la especie correcta (recall) o con cualquier especie (*localizada*). Usamos recall y no AP por grupo porque un falso positivo no tiene tamaño ni oclusión real a la que asignarse. El tamaño se agrupa por área relativa (< 1 %, 1–5 %, > 5 %). Para la oclusión, que CCT-20 no anota, etiquetamos a mano 150 cajas por split de test sobre las imágenes a resolución completa, en tres niveles (bajo < 25 %, medio 25–60 %, alto > 60 % del animal oculto). Re-etiquetamos a ciegas 50 de ellas y medimos la autoconsistencia con κ de Cohen [3].

4.4. Módulo de monitoreo ecológico

Un ecólogo no necesita cajas por foto, sino eventos, especies e individuos por cámara y por periodo. El módulo agrega las detecciones de una cámara con cinco reglas: **R1** una detección cuenta si su confianza es $\geq 0{,}5$; **R2** las fotos separadas por ≤ 60 s forman un evento (se usa la hora de la foto y no el identificador de secuencia, porque en campo solo se tiene la hora); **R3** la especie del evento es la de mayor suma de confianzas; **R4** los individuos de una especie son el máximo de detecciones en un mismo fotograma, no la suma, porque el mismo animal aparece en varias fotos de la ráfaga; **R5** un evento sin detecciones es una activación vacía. Aplicando las mismas reglas a las etiquetas reales obtenemos el reporte que daría una revisión manual, lo que permite medir el error en las unidades que importan al usuario.

5. Resultados

Todas las cifras de test se obtuvieron una sola vez con el modelo final (E3) y umbrales fijados de antemano en validación. La curva F1–confianza de validación alcanza su máximo ($F1 = 0{,}73$) en 0,45 y es prácticamente plana entre 0,4 y 0,6; por eso se usó conf $= 0{,}5$ en los análisis por caja, por tamaño y de monitoreo.

5.1. Ajuste fino y efecto de las imágenes vacías

El split `train` de CCT-20 no contiene imágenes vacías, aunque en campo la mayoría de los disparos lo son. Para medir si conviene enseñarle el fondo al modelo, entrenamos dos YOLOv8n idénticos (640 px, 30 épocas, misma semilla) que solo difieren en que E2 agrega 2 286 imágenes vacías (20 % de las imágenes con animal) tomadas de ubicaciones que no pertenecen a CCT-20, de modo que no filtran información de las cámaras de validación ni de test.

Agregar vacías no solo reduce casi a la mitad los falsos disparos, que era lo esperado, sino que también sube el mAP (+1,8 puntos en mAP@0.5 y +2,8 en mAP@0.5:0.95). Nuestra lectura es que, sin ejemplos

Tabla 1: Efecto de las imágenes vacías en validación (`cis_val` + `trans_val`, 4 213 imágenes). "Falsos disparos": porcentaje de imágenes vacías con al menos una detección de confianza $\geq 0{,}5$. Para E3 esta tasa se midió solo en test (Sección 5.3).

Experimento	mAP@0.5	mAP@0.5:0.95	Falsos disp. cis	Falsos disp. trans
E1: YOLOv8n, sin vacías	0,730	0,486	6,0 %	8,2 %
E2: YOLOv8n, +20 % vacías	0,748	0,514	3,4 %	4,9 %
E3: YOLOv8s, +20 % vacías	0,784	0,539	–	–

negativos de escena completa, el modelo aprende a marcar como animal ciertas texturas del fondo (troncos, rocas, sombras) y esos falsos positivos penalizan la precisión. Con base en esto, el modelo principal E3 se entrenó sobre el conjunto con vacías, cambiando a YOLOv8s y 50 épocas. El entrenamiento se detuvo por *early stopping* en la época 44 (mejor época: 34).

5.2. Desempeño por especie y efecto del desbalance

Tabla 2: AP por especie en test, ordenado por número de cajas en entrenamiento. n: instancias en cada test. Ciervo no aparece en `trans_test`.

		`cis_test`			`trans_test`		
Especie	Cajas train	n	AP@0.5	AP@0.5:0.95	n	AP@0.5	AP@0.5:0.95
zorro (fox)	5	2	0,224	0,184	1	0,000	0,000
tejón (badger)	9	4	0,003	0,002	15	0,141	0,098
ciervo (deer)	44	140	0,504	0,327	0	–	–
zorrillo (skunk)	214	162	0,947	0,629	273	0,818	0,519
roedor (rodent)	264	146	0,811	0,451	26	0,003	0,001
ave (bird)	560	752	0,622	0,327	180	0,094	0,049
lince (bobcat)	684	772	0,863	0,608	1 979	0,703	0,470
perro (dog)	769	725	0,765	0,527	961	0,707	0,476
mapache (raccoon)	1 030	981	0,932	0,616	4 803	0,780	0,509
ardilla (squirrel)	1 037	510	0,724	0,393	782	0,296	0,162
gato (cat)	1 170	1 528	0,933	0,626	1 366	0,293	0,192
coyote	1 371	1 182	0,922	0,699	1 902	0,608	0,427
conejo (rabbit)	2 278	1 559	0,901	0,570	722	0,642	0,342
zarigüeya (opossum)	2 514	4 010	0,948	0,525	4 663	0,885	0,470
mAP		12 473	**0,721**	**0,463**	17 673	**0,459**	**0,286**

La Tabla 2 muestra que el desbalance se nota sobre todo en los extremos. Las dos especies con menos de diez cajas de entrenamiento (zorro y tejón) no pasan de 0,22 de AP@0.5, mientras que las cuatro especies con más de 1 100 cajas superan 0,90 de AP@0.5 en `cis_test`. La correlación de Spearman entre AP@0.5 y número de cajas es $\rho = 0{,}68$ ($p = 0{,}008$, 14 especies) en `cis_test` y baja a $\rho = 0{,}49$ ($p = 0{,}09$, 13 especies) en `trans_test` (Figura 1). Hay que leer con cautela las especies raras: con 2 o 4 instancias de test, un solo acierto o fallo mueve el AP de 0 a 0,5, de modo que lo único que podemos afirmar es que el modelo no aprendió a detectarlas de forma confiable.

El desbalance, sin embargo, no explica las peores caídas en cámaras nuevas. Gato (1 170 cajas) cae de 0,93 a 0,29 y ardilla (1 037) de 0,72 a 0,30, mientras que zorrillo, con solo 214 cajas, se mantiene en 0,82. Esa es la razón de que la correlación se debilite en `trans_test`: fuera de las cámaras de entrenamiento, la cantidad de datos importa menos que la diversidad de escenas en las que aparece cada especie.

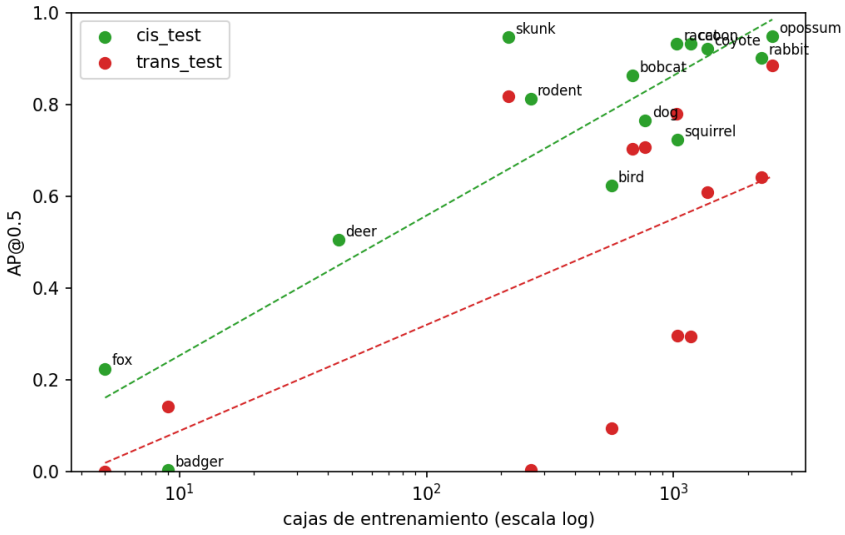


Figura 1: AP@0.5 por especie frente al número de cajas de entrenamiento (escala logarítmica). Verde: `cis_test`; rojo: `trans_test`. Las líneas discontinuas son solo una guía visual.

5.3. Generalización a ubicaciones nuevas y aprendizaje del fondo

Entre `cis_test` y `trans_test` el mAP@0.5 cae de 0,721 a 0,459 (−26 puntos) y el mAP@0.5:0.95 de 0,463 a 0,286. La caída viene sobre todo del recall (0,66 → 0,42); la precisión baja menos (0,87 → 0,67) y la tasa de falsos disparos en las vacías de test casi no cambia (3,6 % en cis frente a 4,3 % en trans). En otras palabras, en una cámara nueva el modelo deja de ver animales mucho más de lo que inventa.

Para saber si el modelo aprendió el fondo de las cámaras de entrenamiento como atajo, aplicamos dos pruebas sobre 300 imágenes con un solo animal por split.

Prueba de borrado. Tapamos la caja del animal (con un margen de 15 %) con el color medio de la imagen y volvimos a detectar. Si el modelo dependiera del contexto, debería seguir prediciendo la especie aunque el animal ya no esté. No ocurrió: en ningún caso la misma especie reapareció en la zona borrada, y solo hubo 1 de 300 casos en `cis_test` (un conejo) y 2 de 300 en `trans_test` (dos ardillas) en que la especie se predijo en otra parte de la escena (los tres casos se muestran en el Apéndice B). El resultado es el mismo si nos quedamos solo con las imágenes que el modelo había detectado bien al principio (250 en cis y 210 en trans).

EigenCAM. Implementamos EigenCAM sobre la salida del bloque SPPF (capa 9, final del *backbone*) y medimos qué fracción de la energía del mapa cae dentro de las cajas reales, normalizada por el área que ocupan (*lift*; 1 equivale a no preferir al animal sobre el resto de la imagen). El lift mediano fue de 0,05–0,08 en todos los grupos, incluidas las detecciones correctas (Tabla 3). Visualmente, el mapa se concentra en estructuras grandes de la escena (la franja de vegetación del horizonte, el borde del camino) y no en el animal (Figura 2).

Las dos pruebas parecen contradecirse, pero responden preguntas distintas. EigenCAM proyecta las activaciones sobre su primera componente principal, es decir, sobre la dirección de mayor varianza espacial; en una cámara fija, esa varianza la domina la estructura de la escena, no un animal que en esta muestra ocupa entre el 3 y el 14 % de la imagen. El mapa muestra entonces que la representación del *backbone* está dominada por el fondo, pero no que la decisión de la cabeza de detección dependa de él, y la prueba de borrado indica que no depende. Concluimos que la caída cis→trans no se explica por un atajo del tipo “esta cámara ⇒ esta especie”, sino por animales que el modelo no reconoce cuando cambian el fondo, el ángulo y la iluminación. Esto es coherente con que la pérdida sea de recall y no de precisión.

Tabla 3: EigenCAM en la capa 9: fracción de energía dentro de las cajas reales, fracción de área que ocupan y lift mediano (150 imágenes por split).

Split	Detección correcta	n	Energía en cajas	Área de cajas	Lift mediano
`cis_test`	sí	124	0,022	0,048	0,059
	no	26	0,060	0,068	0,083
`trans_test`	sí	97	0,022	0,033	0,047
	no	53	0,111	0,137	0,085

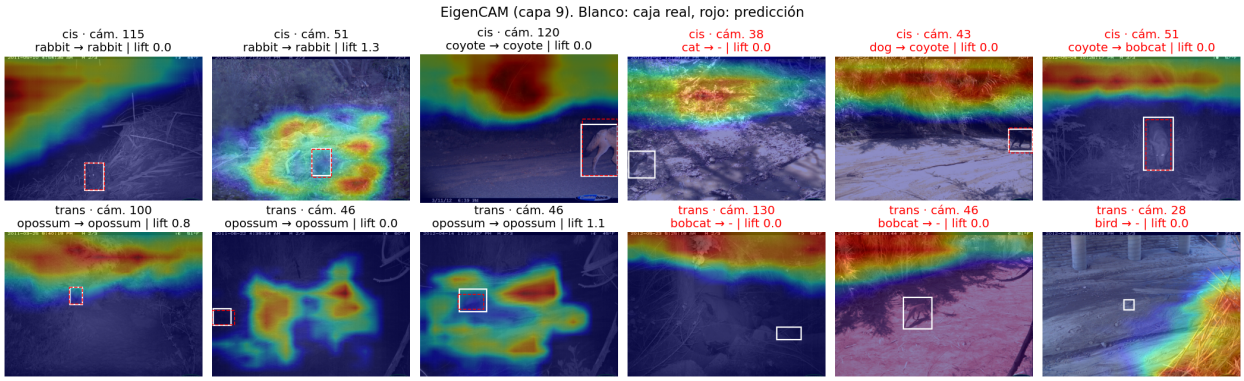


Figura 2: EigenCAM (capa 9). Fila superior: `cis_test`; inferior: `trans_test`. Blanco: caja real; rojo discontinuo: predicción. Títulos en rojo: detecciones fallidas.

La distribución de las especies entre las cámaras de entrenamiento apoya esta lectura, aunque solo de forma parcial. Las tres especies que más caen en `trans_test` son las más concentradas en train: el 90 % de las cajas de roedor, el 88 % de las de gato y el 81 % de las de ave provienen de solo dos de las diez cámaras (el 74 % de los gatos, de la cámara 38). El modelo vio a esas especies casi siempre sobre el mismo fondo y con el mismo ángulo. En el otro extremo, zorrillo está repartido en siete cámaras (las dos principales suman el 57 %) y conserva 0,82 de AP@0.5. Sin embargo, la relación no es general: considerando las 11 especies con datos suficientes, la correlación entre concentración y caída es débil ($\rho = 0{,}33$, $p = 0{,}32$). Ardilla está bien repartida y aun así cae 43 puntos, y zarigüeya está concentrada pero su abundancia la protege.

5.4. Robustez al tamaño y a la oclusión

Medimos el recall por caja (IoU $\geq 0{,}5$ y especie correcta) y, por separado, si la caja fue *localizada* con cualquier especie. Así se distinguen los errores de localización de los de clasificación. Los cortes de tamaño se fijaron según el área relativa de la caja: pequeña < 1 %, mediana 1–5 % y grande > 5 %.

Tabla 4: Recall y fracción localizada por tamaño de caja (conf $\geq 0{,}5$).

	`cis_test`			`trans_test`		
Tamaño	n	Recall	Localizada	n	Recall	Localizada
Pequeña (< 1 %)	1 149	0,560	0,591	1 924	0,346	0,376
Mediana (1–5 %)	8 929	0,844	0,895	12 183	0,713	0,788
Grande (> 5 %)	2 395	0,823	0,869	3 566	0,462	0,637

Los animales pequeños son el punto débil esperado: al reescalar la imagen de 1024 px a 640 px, una caja que ocupa el 1 % del área queda reducida a unos 60×50 píxeles, y en esos casos casi todos los fallos son de localización (recall y localizada casi iguales). Lo inesperado está en las cajas grandes de `trans_test`

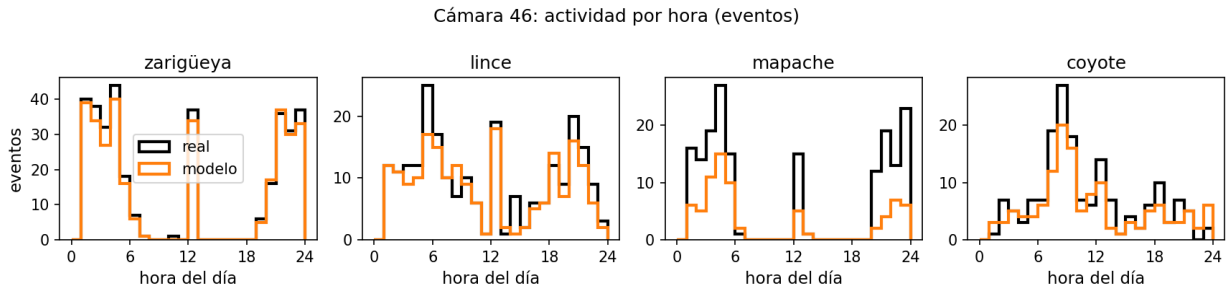


Figura 3: Cámara 46 (no vista en entrenamiento): eventos por hora del día para las cuatro especies más frecuentes, según el modelo (naranja) y según la revisión manual (negro).

(Apéndice C): el recall cae a 0,46, por debajo del de las medianas, mientras que el 64 % de esas cajas sí se localiza. Es decir, el modelo encuentra al animal pero se equivoca de especie.

Para entender por qué, revisamos las 625 cajas grandes de `trans_test` localizadas con la especie equivocada. El 80 % de ellas toca el borde de la imagen (a menos del 1 % del marco), frente al 34 % de las cajas grandes clasificadas bien; es decir, son animales tan cerca del lente que quedan cortados por el encuadre. También son más nocturnas (71 % frente a 56 %), lo que suma la pérdida de color del infrarrojo. Sin la silueta completa, el modelo recurre a la especie más parecida: la confusión dominante es mapache → coyote (251 cajas, el 40 % de estos errores), seguida de gato → coyote (65) y coyote → gato (56).

Oclusión. *[Pendiente: recall por nivel de oclusión (bajo $<$ 25 %, medio 25–60 %, alto $>$ 60 %) sobre 150 cajas por split etiquetadas manualmente, y κ de autoconsistencia sobre 50 cajas re-etiquetadas a ciegas.]*

5.5. Módulo de monitoreo en una cámara no vista

Aplicamos el módulo de monitoreo (reglas R1–R5, sección 4) a la cámara 46 de `trans_test`, la de más fotos entre las cámaras que el modelo nunca vio: 6 276 fotos entre mayo de 2011 y junio de 2012, agrupadas en 1 383 eventos. El mismo procedimiento aplicado a las etiquetas reales da el reporte que se obtendría revisando las fotos a mano, y sirve de referencia (Tabla 5 y Figura 3).

Tabla 5: Reporte de la cámara 46: modelo frente a revisión manual.

Indicador	Modelo	Real
Fotos vacías	55,5 %	11,6 %
Eventos vacíos	31,8 %	16,1 %
Especies registradas	10	12
Acierto de especie por evento	75,0 %	
Eventos con animal marcados vacíos	221 de 1 161 (19 %)	
Eventos vacíos marcados con animal	3	
% nocturno zarigüeya / mapache	0,87 / 0,89	0,87 / 0,90
% nocturno zorrillo / lince	0,76 / 0,50	0,81 / 0,55
Eventos zarigüeya / lince / coyote	319 / 203 / 134	344 / 232 / 168
Eventos mapache	74	175
Eventos gato / ave / roedor	55 / 3 / 0	5 / 25 / 14

Agrupar en eventos compensa buena parte de los errores por foto: aunque el modelo deja sin detección a más de la mitad de las fotos, basta con que acierte en una foto de la ráfaga para registrar el evento, y la diferencia en eventos vacíos se reduce a la mitad (31,8 % frente a 16,1 %). Para las especies abundantes, los indicadores que usaría un equipo de manejo se reproducen bien: la fracción de eventos nocturnos difiere a lo sumo en 0,05 para zarigüeya, mapache, zorrillo y lince, y el número de eventos de zarigüeya, lince y coyote

se subestima entre un 7 y un 20 %. El mapache es la excepción: la proporción nocturna se conserva, pero el modelo registra 74 eventos frente a 175 reales. De los 175 eventos reales de mapache, 74 quedan marcados como vacíos y 17 como coyote, la misma confusión observada en las cajas grandes. Las especies pequeñas (ave y roedor) casi desaparecen del reporte, en línea con el recall de las cajas pequeñas: 21 de los 25 eventos de ave y 12 de los 14 de roedor se marcan como vacíos. En cambio, el gato aparece sobrerrepresentado (55 eventos frente a 5 reales). La matriz de confusión por evento muestra de dónde vienen esos "gatos": 24 son linces, 16 coyotes y 8 mapaches. En la práctica, el módulo es útil para estimar patrones de actividad de las especies comunes, pero no para inventarios de especies raras o pequeñas, y no permite prescindir de la revisión humana de los eventos que marca como vacíos, donde se esconde el 19 % de los eventos con animal.

6. Consideraciones éticas

Un sistema como este puede ayudar a la conservación, pero también puede causar daño si se usa sin cuidado. Identificamos cuatro riesgos concretos.

Decisiones de manejo sesgadas contra las especies raras. El error del modelo no se reparte por igual: las especies con pocos ejemplos o concentradas en pocas cámaras (tejón, zorro, roedor, ave) son las que peor se detectan, y suelen ser también las de mayor interés para la conservación. Un reporte automático que las omite podría llevar a concluir que una especie desapareció de una zona cuando solo es invisible para el modelo. Por eso los reportes deben indicar la incertidumbre por especie y no deberían usarse para inventarios de especies raras sin revisión humana.

Falsa sensación de completitud. En la cámara 46, el 19 % de los eventos con animal se marcan como vacíos. Si la herramienta se usa para descartar fotos sin mirarlas, esos eventos se pierden para siempre. Recomendamos un esquema con un humano en el circuito: revisar una muestra aleatoria de los eventos marcados como vacíos y priorizar la revisión de las detecciones de baja confianza.

Ubicación de especies sensibles y privacidad. Los metadatos de una cámara trampa (coordenadas, horarios de actividad) pueden facilitar la caza furtiva si se publican junto con las detecciones de especies amenazadas. Además, las cámaras fotografían a personas (guardaparques, excursionistas, comunidades locales) que no dieron su consentimiento. Un despliegue responsable debería generalizar las coordenadas antes de publicar, restringir el acceso a los datos crudos y detectar y difuminar a las personas antes de compartir imágenes.

Transferencia a otros ecosistemas. CCT-20 proviene de un único ecosistema del suroeste de Estados Unidos. Nuestros resultados muestran que incluso entre cámaras de la misma región el mAP cae 26 puntos, de modo que usar este modelo, por ejemplo, en la Amazonía o en los Andes peruanos, con especies y escenas distintas, requeriría datos locales y una nueva evaluación. Presentarlo como solución general sería engañoso. El costo ambiental del entrenamiento fue bajo (unas pocas horas de una GPU T4), pero no nulo.

7. Limitaciones y conclusiones

Este trabajo tiene limitaciones claras. Cada configuración se entrenó una sola vez con una semilla, por lo que diferencias pequeñas (como el +1,8 de mAP de E2 sobre E1) no tienen intervalo de confianza. Las especies con menos de 10 instancias de test no permiten conclusiones. El análisis de oclusión se basa en etiquetas de un solo anotador, y EigenCAM sobre el *backbone* resultó poco informativo sobre la decisión del detector; un mapa basado en gradientes de la puntuación de cada caja sería el siguiente paso.

Con esas reservas, concluimos que: (1) incluir imágenes vacías de otras ubicaciones es una decisión barata que reduce a la mitad los falsos disparos y mejora el mAP; (2) en cámaras nuevas el detector pierde sobre todo recall, no precisión, y la caída golpea más a las especies vistas en pocas cámaras; (3) el detector no predice especies a partir del fondo, sino que deja de reconocer animales cuando el fondo cambia; y (4) agregado en eventos, el modelo es útil para estimar patrones de actividad de especies comunes en una cámara nueva, pero no reemplaza la revisión humana para inventarios completos.

Reproducibilidad. El código, el cuaderno de entrenamiento y las instrucciones están en `https://github.com/roshan0297/rnc-pregunta3-parcial`. Todo el flujo, desde la descarga hasta los análisis, se ejecuta con `pip install -r requirements.txt` seguido de `python run_all.py`.

Referencias

[1] Sara Beery, Grant Van Horn, and Pietro Perona. Recognition in terra incognita. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 456–473, 2018.

[2] Sara Beery, Dan Morris, and Siyu Yang. Efficient pipeline for camera trap image review. *arXiv preprint arXiv:1907.06772*, 2019.

[3] Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1): 37–46, 1960.

[4] Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.

[5] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics YOLOv8. `https://github.com/ultralytics/ultralytics`, 2023.

[6] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.

[7] Mohammed Bany Muhammad and Mohammed Yeasin. Eigen-CAM: Class activation map using principal components. In *International Joint Conference on Neural Networks (IJCNN)*, 2020.

[8] Mohammad Sadegh Norouzzadeh, Anh Nguyen, Margaret Kosmala, Alexandra Swanson, Meredith S. Palmer, Craig Packer, and Jeff Clune. Automatically identifying, counting, and describing wild animals in camera-trap images with deep learning. *Proceedings of the National Academy of Sciences*, 115(25):E5716–E5725, 2018.

[9] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017.

[10] Michael A. Tabak, Mohammad S. Norouzzadeh, David W. Wolfson, et al. Machine learning to classify animal species in camera trap images: Applications in ecology. *Methods in Ecology and Evolution*, 10(4):585–590, 2019.

A. Elección del umbral de confianza

La curva se calcula sobre validación (`cis_val` + `trans_val`); el umbral no se ajustó en test.

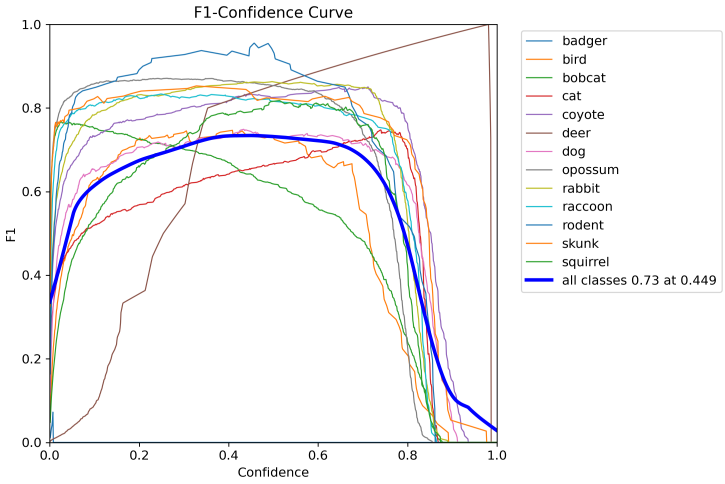


Figura 4: Curva F1–confianza de E3 en validación. El máximo promedio ($F1 = 0{,}73$) se alcanza en 0,45 y la curva es plana entre 0,4 y 0,6.

B. Ejemplos de la prueba de borrado

De las 600 imágenes de la prueba (300 por split), solo en estas tres el modelo volvió a predecir la especie del animal borrado, y siempre fuera de la zona tapada.



Figura 5: Casos en que, tras borrar al animal (rectángulo punteado blanco), el modelo vuelve a predecir la misma especie (rojo) en otra parte de la escena.

C. Recall por tamaño de caja

Versión gráfica de la Tabla 4; bajo cada par de barras se indica el número de cajas en `cis_test` / `trans_test`.

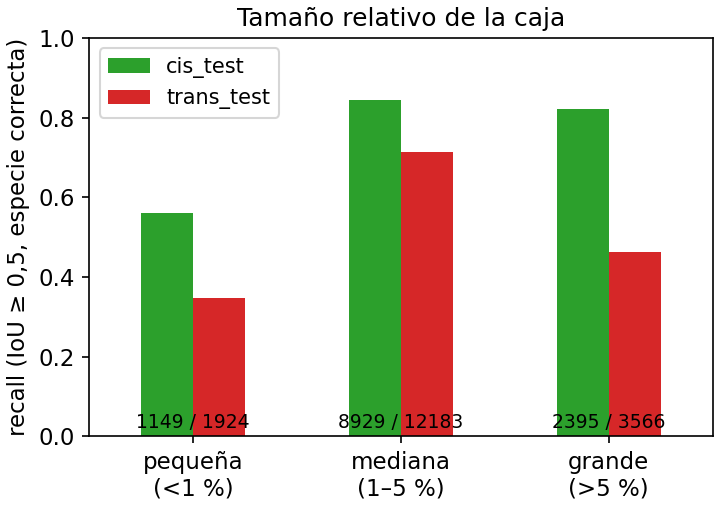


Figura 6: Recall (IoU $\geq$ 0,5, especie correcta) por tamaño de caja en `cis_test` (verde) y `trans_test` (rojo).